

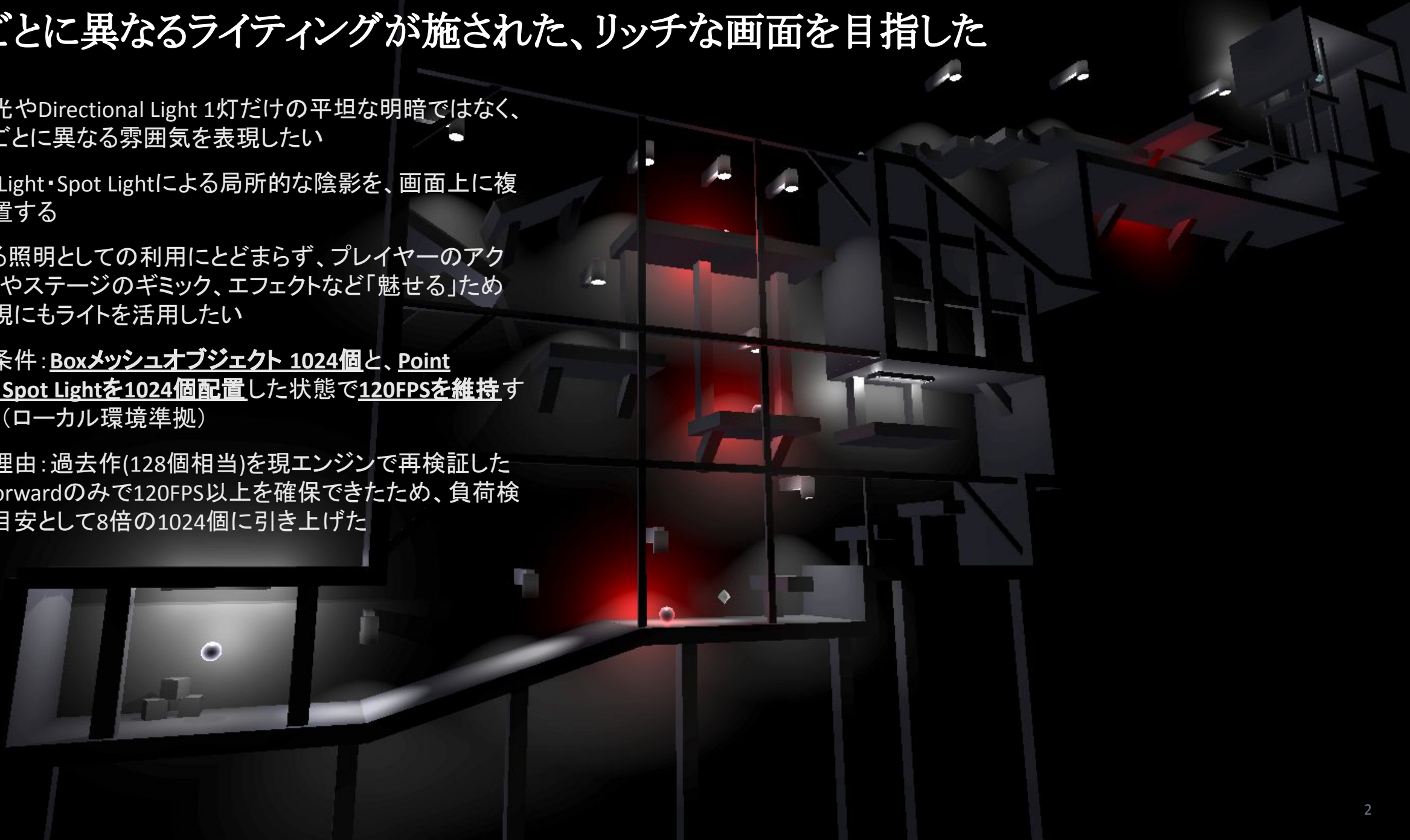
多数のライトを配置しても、軽い画面をつくる

Forward+ タイルカリングの実装と検証計画

鈴木 色音

空間ごとに異なるライティングが施された、リッチな画面を目指した

- 環境光やDirectional Light 1灯だけの平坦な明暗ではなく、場所ごとに異なる雰囲気表現したい
- Point Light・Spot Lightによる局所的な陰影を、画面上に複数配置する
- 単なる照明としての利用にとどまらず、プレイヤーのアクションやステージのギミック、エフェクトなど「魅せる」ための表現にもライトを活用したい
- 達成条件：Boxメッシュオブジェクト 1024個と、Point Light・Spot Lightを1024個配置した状態で120FPSを維持すること(ローカル環境準拠)
- 設定理由：過去作(128個相当)を現エンジンで再検証した所、Forwardのみで120FPS以上を確保できたため、負荷検証の目安として8倍の1024個に引き上げた

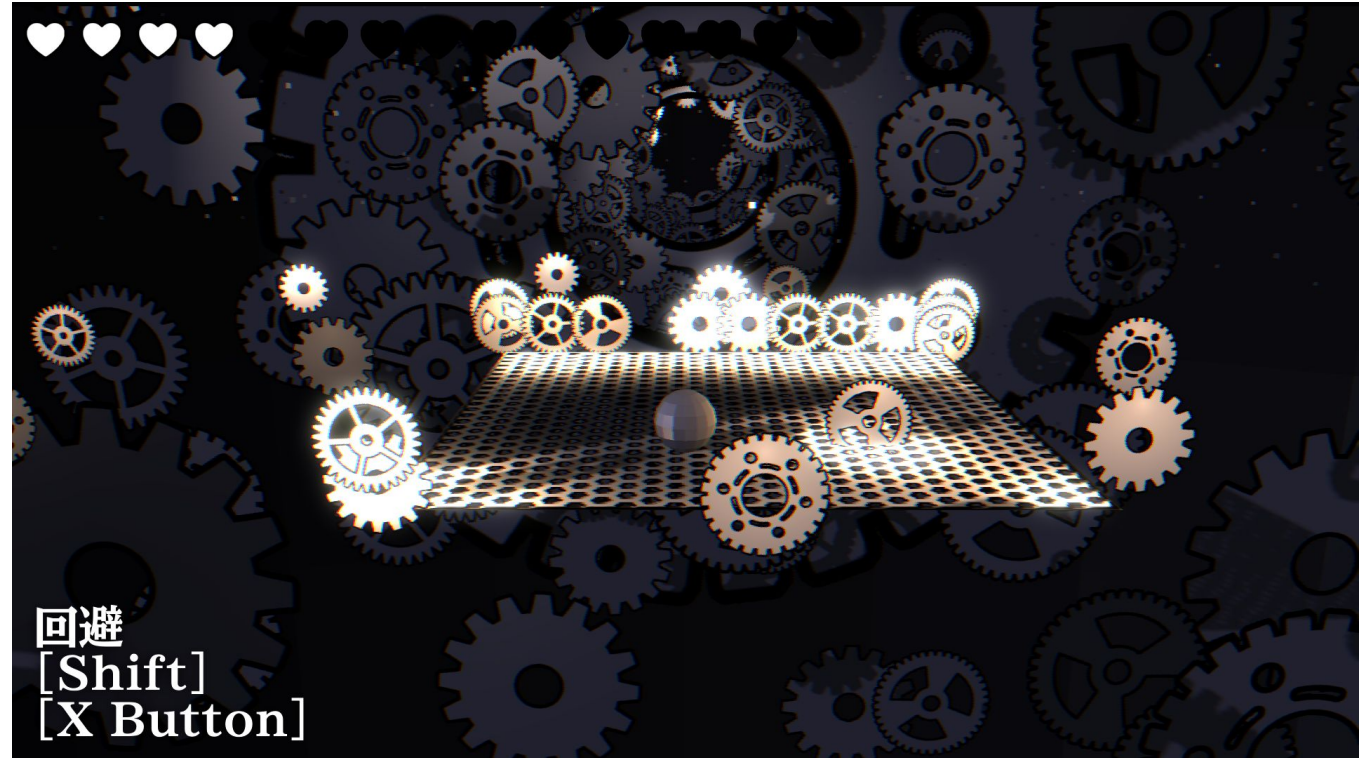


ライトを増やすほど、フォワードレンダリングの負荷が増える

- 通常のフォワードレンダリングでは、オブジェクトを描画する際にそのオブジェクトが占める各ピクセル(描画対象のフラグメント)ごとに、シェーダーが全ライトを参照する
- ライトごとに方向・距離の計算、減衰、Spot Lightのコーン判定、加算処理が発生する
- 負荷は「シェーディングされるピクセル数 × 参照するライト数」に応じて増加する
- そのままメッシュとライトを1024個配置すると参照回数が増えすぎ、FPSが低下する懸念があった

過去の制作でライト数によるパフォーマンス低下に悩まされた経験があった

- 過去の制作で512個以上の板ポリと128個のポイント+スポットライトの多用でパフォーマンスが低下した経験があった
- エンジンの組み方やビルド構成にも難がありそれもだいぶ影響してはいたが、その難とライトが増えることによるパフォーマンス低下が重なり余計にパフォーマンスが低下してしまった
- 現在のエンジン環境で過去作相当の条件を再検証した所、Forwardレンダリングのみで120FPS以上を余裕を持って確保できることを確認。当時の低下はライト数自体よりビルド構成やエンジンの組み方の課題が主要因だったと判断した
- この気づきを踏まえ、目標を過去の8倍にあたる1024個へ引き上げて検証した(今回のメインはレンダリング部分)



過去に制作したゲームのスクリーンショット。
512枚以上の板ポリと、ポイント・スポットライトが計128個配置されている。

既存のフォワードパイプラインを活かせる Forward+ タイルカリングを採用

評価項目	ディファードレンダリング	Forward+ タイルカリング
既存パイプラインからの移行コスト	高い(描画パイプラインの作り直しが必要)	低い(既存のフォワードにカリングを追加)
ライトカリングの要否	必要(カリングなしではピクセル数×ライト数のまま)	必要(タイルカリングで対応)
目的(ライト参照数の削減)への効き方	間接的	直接的

採用理由

本作はすでにフォワードレンダリングで構築済みのため、既存実装を活かしつつ最小の変更で効果を検証できる Forward+タイルカリングを採用した。

シーン特性を踏まえ、深度プリパスは見送りタイルカリングのみ先行実装

- 深度プリパスが効くのは、奥行き方向に大量のオブジェクトが重なる場面
- 本作はそのような場面が少ないため、優先度は低いと判断した
- まずタイルカリングのみを実装し、**目標(1024個配置で120FPS)**を満たせない場合に**深度プリパスの追加を検討** する

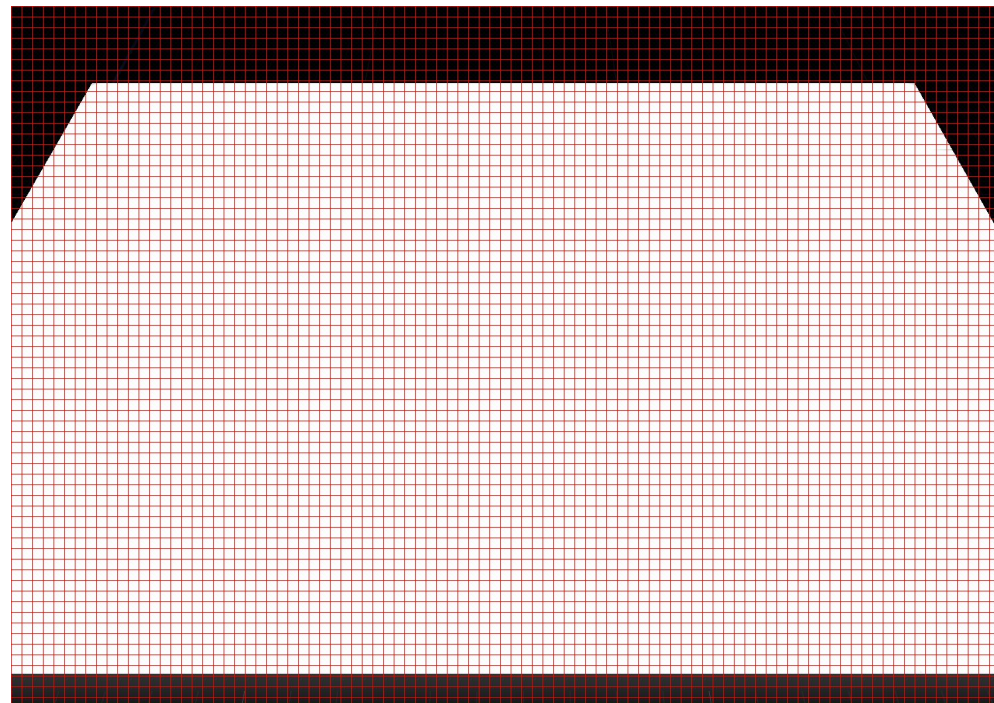
Compute Shaderで画面を16×16タイルに分割し、影響ライトだけ絞り込む

Compute Shader (LightCullingCS.hlsl)

- 画面を16×16ピクセルのタイルに分割し、各タイルに影響するライトのインデックスを書き込む
- 各ライト(Point/Spot)の包含球からビュー空間AABBを計算し、スクリーン空間へ射影して外接矩形を求める
- タイルの矩形とライトの射影矩形のAABB重なり判定でカリングする
- 1タイルあたり最大1024ライトを格納。SpotLightは最上位ビットで識別しPointLightと区別する

CPU側の管理 (Renderer.cpp)

- 描画先(ScreenBuffer)とパイプラインの組ごとにカリングを実行する
- カリング結果バッファとシャドウマップを描画パイプラインへバインドする



画面を16×16ピクセルのタイルに分割した図

深度プリパスなしのため、遮蔽されたライトも一部計算に含まれる

- 深度プリパスなし → 手前のオブジェクトに完全に遮蔽されたライトも一部計算対象になる(保守的な判定)
- タイルカリング自体のCompute Shader実行コストが毎フレーム発生する
- 自作エンジンの実装として、1タイルあたりのライト格納数を1024に制限している。極端にライトが密集した場合はクランプされる

計測条件を固定し、同一条件で比較する

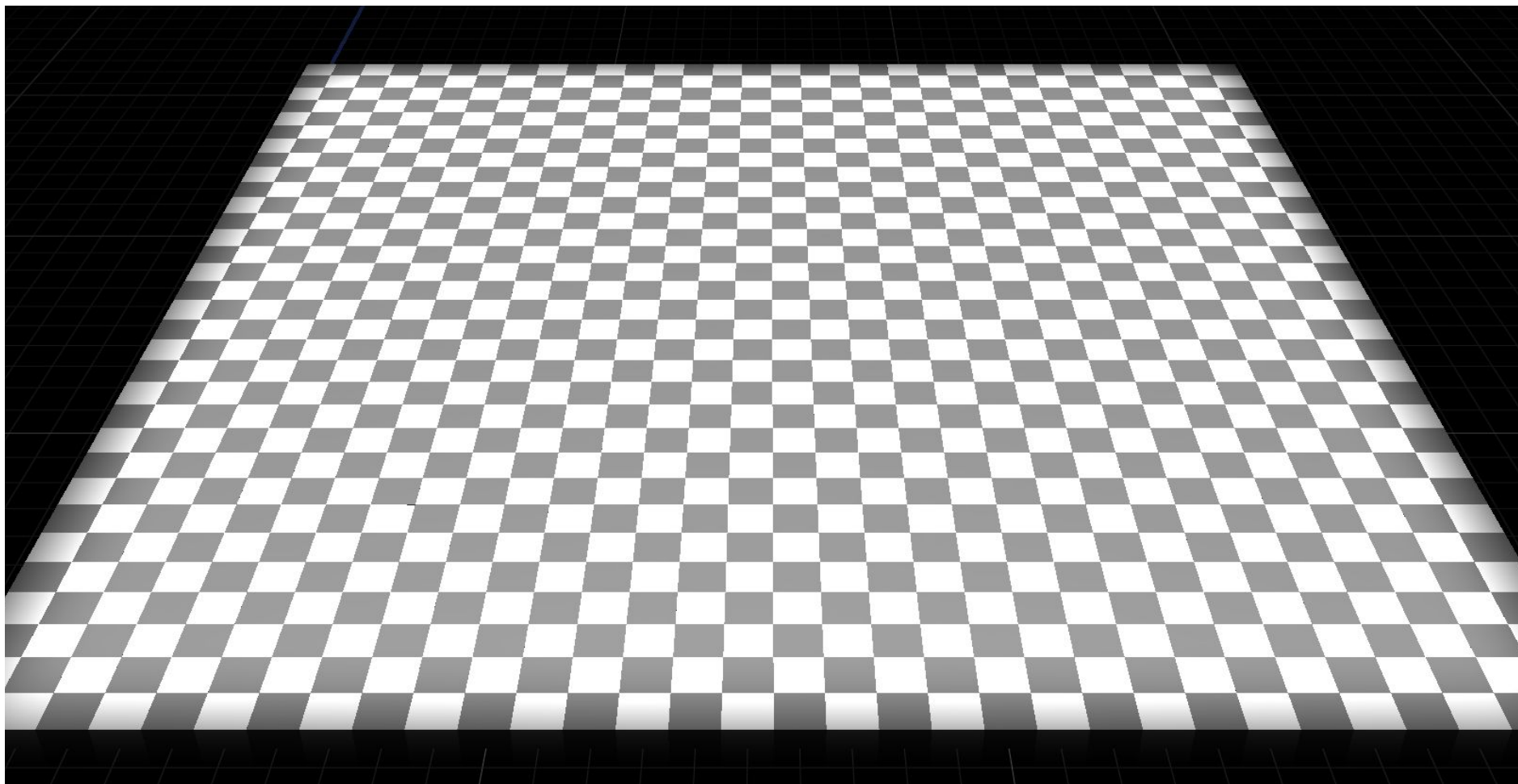
固定する項目	内容
CPU / GPU	Ryzen 7 7745HX / RTX 4070 Laptop
解像度 / ディスプレイ最大FPS	1920 × 1080 / 240fps
ビルド構成	Development (Optimization: MaxSpeed)
シーン	テスト用の専用シーン。隙間なく敷き詰められた1024個のBoxメッシュオブジェクトと、カメラ位置・向き、実行時間、入力なしの状態を固定
ライト	ライト半径4のポイントライトを使用し、影響範囲や配置パターンを固定
計測方法	エンジン内プロファイラで300フレームの間で計測を行い、FPSと描画時間(※)の平均値を取る

※FPSは描画時間だけでなく更新時間も含めた上での計測値、描画時間はエンジン内の描画部分のコードだけでの計測値。

現在のステータス

就活作品ではまだメッシュとライトが 1024個配置されているシーンが無いため、テスト用のシーンでの計測。将来的に就活作品のほうでも数百個～数千個のオブジェクトとライトを配置し計測予定。

テストシーンは 1024個のBoxメッシュを隙間なく敷き詰めて構成



テスト用のシーン。1024個のBoxメッシュが隙間なく敷き詰められている。
ここでは見やすくするために、互い違いに色を変えている。

計測は3ステップで実施する

1

タイルカリングの有効／無効を切り替え、同一シーンで描画時間と FPSを比較する

2

ライト数を64 → 128 → 256 → 512 → 1024と段階的に増やし、それぞれ記録する

3

エンジン内プロファイラから FPSと描画時間をスクリーンショットで記録する

計測の結果、1024個のBoxとライトで 120fps以上を維持することができた

以下はそれぞれの計測時の早見表である。

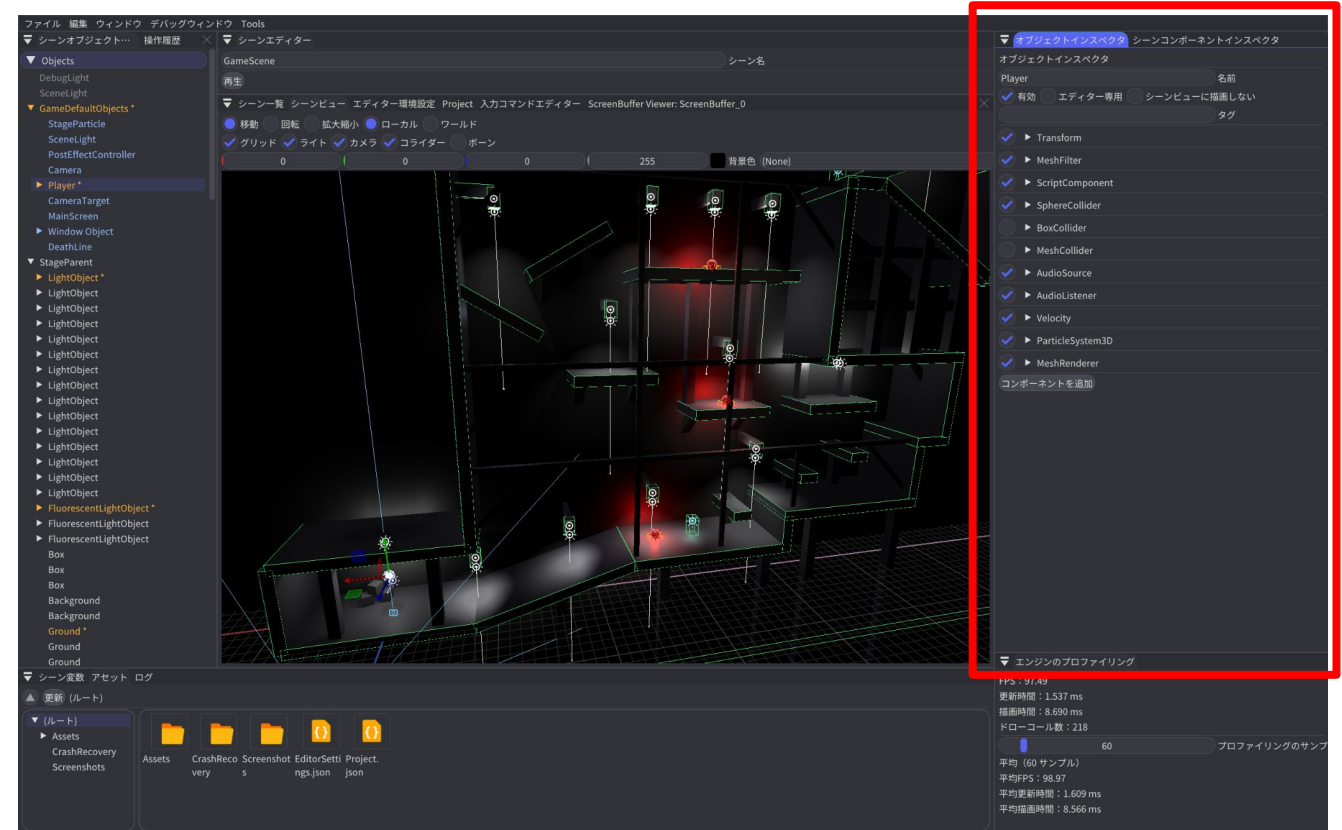
ライト数(個) レンダリング方式	64	128	256	512	1024
Forward	描画時間: 3.356ms FPS: 226.72 	描画時間: 5.854ms FPS: 141.20 	描画時間: 11.073ms FPS: 79.17 	描画時間: 19.111 FPS: 47.82 	描画時間: 37.263ms FPS: 25.02 
Forward+	描画時間: 3.114ms FPS: 239.83 	描画時間: 2.990ms FPS: 239.13 	描画時間: 3.195ms FPS: 231.07 	描画時間: 4.201ms FPS: 172.90 	描画時間: 6.350ms FPS: 120.96 

シーン特性に合わせて導入範囲を選ぶことの重要性を学んだ

- ディファードへの全面移行ではなく、既存パイプラインに追加できるタイルカリングを選んだ
- 深度プリパスのような高精度な手法もあるが、奥行き为重なりが少ない本作の特性を踏まえ、タイルカリングのみで目標を満たせるかを先に確認する方針にした
- 今後ライト数がさらに増える場合は、深度プリパスの追加を検討する
- Compute Shaderによる事前計算のアプローチは、パーティクルなど他のGPU処理にも応用できると考えている

多数のギミックに対応するため、継承ではなくコンポーネント指向を採用

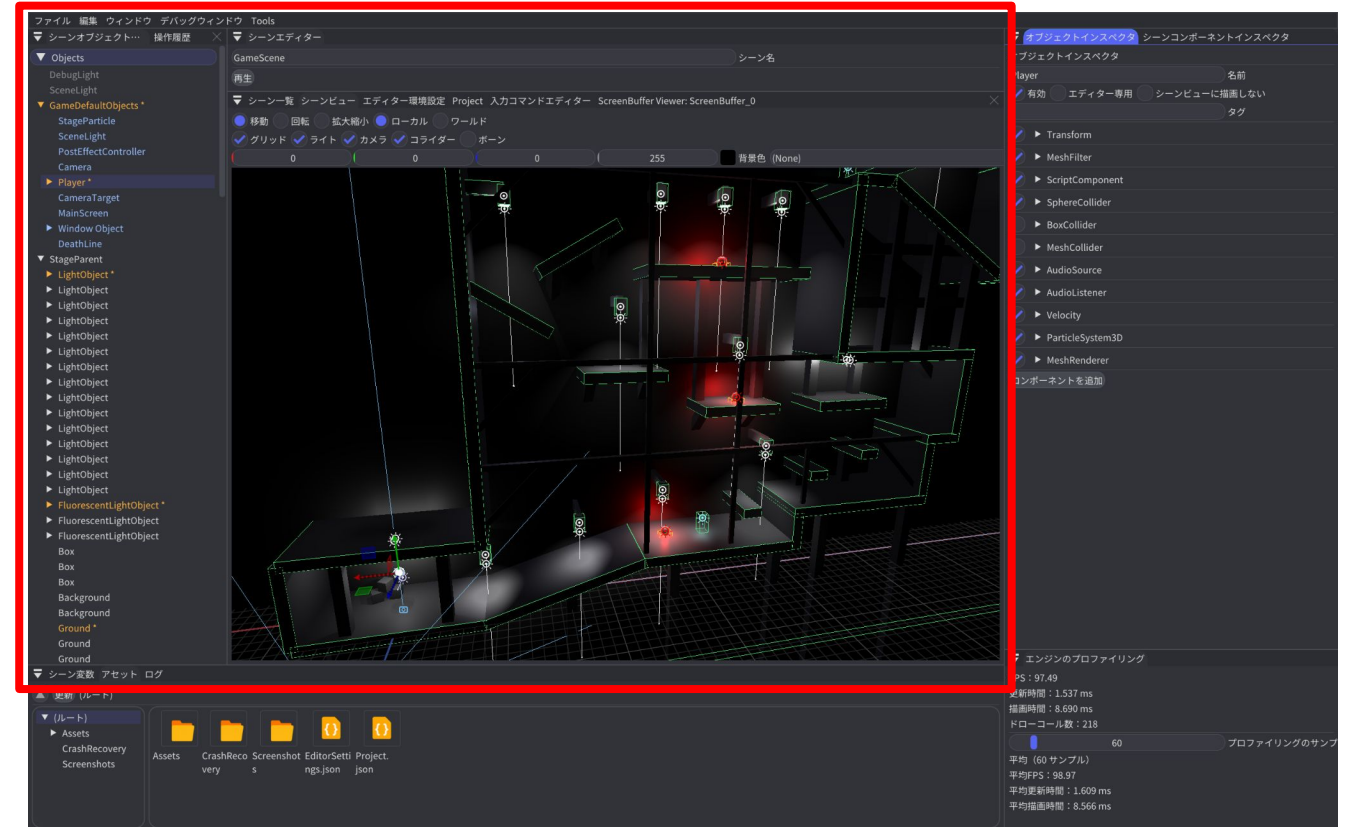
- 本作は自作エンジンによる個人制作
- 多数のステージギミックを実装するにあたり、継承ベースのオブジェクト指向では拡張性に欠け、自由にステージを組めないと判断した
- 機能をコンポーネント単位で組み合わせる設計にすることで、ギミックごとに必要な機能だけを付け外しできるようにした



エディター画面のスクリーンショット。インスペクターで複数のコンポーネントを組み合わせる機能で構成している。

配置の視覚確認とギミック調整の高速化のため、レベルエディターを自作

- 本作は自作エンジンによる個人制作
- ステージ制作ではオブジェクト配置を視覚的に確認しながら組めないと効率が悪く、独自にレベルエディターを実装した
- ギミックを調整するたびにC++側のコードを変更してビルドし直すのでは、効率が悪すぎるという課題もあった
- ヒエラルキー・ビューポート・インスペクター・プロファイラを1つのウィンドウにまとめ、配置とパラメータ調整をその場で確認できるようにした



エディター画面のスクリーンショット。ヒエラルキーとシーンビューで、オブジェクトの配置を視覚的に確認しながら編集できる。

AngelScript導入で、ギミック実装の試行錯誤を高速化した

- 課題: C++は変更のたびに再コンパイルが必要(小変更10~30秒、大変更3~10分)。ギミック1つの追加だけで1~2時間のロスが発生していた
- 比較: Lua(構文差による学習コスト)、C#(.NETランタイム導入コスト)と比べ、C++に近い構文でランタイム不要なAngelScriptを採用
- 実装範囲: ScriptComponent(Unity風ライフサイクル)、型バインディング、SerializeFieldでのインスペクター連携、エディター拡張(キーフレームアニメーションエディター、WFCタイルエディター等)
- デバッグ環境: ローカルネットワークを介してVSCodeと連携する仕組みを整備。VSCode上でブレークポイントを設定し、スクリプトの処理や変数の値を確認できる
- 結果: 6個のギミックを5日間で実装
- 今回1024個のBoxメッシュとポイントライトを配置する用途でもAngelScriptを利用している

代償(未計測)

実行速度はC++より遅い(具体値は未計測、今後 C++版と比較予定)。

主要機能は実装済み。残るは AngelScriptの性能比較と実践シーンでの計測

区分	内容
実装済み・動作確認済み	Forward+タイルカリング／コンポーネントシステム／エディター／AngelScript統合／VSCode連携によるスクリプトデバッガー／ポストプロセス(Bloom、FXAA 等 計15種)
実装済み・性能未計測	AngelScriptの実行速度比較(C++版との差)
今後の予定	深度プリパスの導入検討／実際の作品で数百個～数千個のオブジェクトとライトを配置した際のパフォーマンスの計測